

Logika Paletyzatora – Opis Algorytmu

1. Założenia i Model Danych

1.1 Referencyjna Lista Zleceń

- PLC **nie** inkrementuje ślepo licznika NN. Dopuszczalna kolejność paneli w palecie jest zdefiniowana przez z góry załadowaną do PLC tablicę (`ordersList`).
- Każde zlecenie w tablicy to obiekt { `zlecenie`, `panels[]` }, gdzie `panels[]` zawiera oczekiwane numery NN w prawidłowej kolejności układania.

1.3 Podział jednego zlecenia na kilka fizycznych palet

Ze względu na gabaryty jedno zlecenie (np. 0000000001) może być rozbite na kilka fizycznych palet (np. paleta A: panele 01–06, paleta B: panele 07–14). Dla sterownika każda pod-paleta to osobny wpis w `ordersList`.

2. Stan Sterownika PLC – Zmienne

<code>ordersList[]</code>	- lista wszystkich zleceń załadowanych do PLC
<code>activeOrderIdx</code>	- indeks aktywnej palety w <code>ordersList</code>
<code>nextOrderIdx</code>	- indeks następnej palety (= <code>activeOrderIdx + 1</code>)
<code>expectedInActiveIdx</code>	- indeks oczekiwanego panela wewnątrz aktywnej
<code>a1[]</code>	- bufor daleki (stos LIFO/kolejka, max. rozmiar <code>maxA1</code>)
<code>a2[]</code>	- bufor bliski (stos LIFO/kolejka, max. rozmiar <code>maxA2</code>)
<code>currentPaleta[]</code>	- panele ułożone na bieżącej fizycznej palecie

Bufory są indeksowane jak tablice: `a1[0]` = **dół** (najstarsza warstwa), `a1[last]` = **góra** (ostatnia dodana warstwa).

3. Okno Procesowe (Windowing)

Sterownik w danej chwili „widzi” tylko **dwa** zlecenia:

Slot	Rola
Paleta_Aktywna (ordersList[activeOrderIdx])	Paleta aktualnie układana; zawsze szukamy panela panels[expectedInActiveIdx]
Paleta_Następna (ordersList[nextOrderIdx])	Bufferowanie z wyprzedzeniem; jej panele mogą trafiać do A1 / A2

Panele spoza tego dwuelementowego okna są traktowane jako **ODRZUT**.

4. Pętla Główna – Klasyfikacja Skanowanego Panela

Dla każdego zeskanowanego kodu QR:

```
parse(QR) → { nn, zlecenie }

if zlecenie == Paleta_Aktywna.zlecenie:
    if nn == panels[expectedInActiveIdx]:           → [4A] TRAFIA NA
    else:                                           → [4B] BUFOROWANI
elif zlecenie == Paleta_Następna.zlecenie:        → [4B] BUFOROWANI
else:                                              → [4C] ODRZUT
```

4A – Panel trafia na paletę

1. Dołącz nn do currentPaleta[].
2. Zwiększ expectedInActiveIdx.
3. Jeśli expectedInActiveIdx >= len(pannels) → wywołaj [6] SHIFT.
4. W przeciwnym razie wywołaj [5] ODZYSK.

4B – Buforowanie

```
if len(a1) < maxA1: a1.push(panel) → log: "→ Bufor A1"  
elif len(a2) < maxA2: a2.push(panel) → log: "→ Bufor A2"  
else: → BLOKADA (oba bufory pełne)
```

4C – Odrzut

Panel logujemy jako 🚫 ODRZUT: Panel poza oknem Aktywna/Następna. Licznik rejectedCount rośnie. Produkcja toczy się dalej (panel nie wjeżdża na linię).

5. Procedura Odzysku (recovery())

Wywoływana po każdym ułożeniu panela na palecie (kroki 4A i 6). Działa jako pętla while (found):

Dopóki znaleziono dopasowanie:

```
targetNN = panels[expectedInActiveIdx]  
activeZl = Paleta_Aktywna.zlecenie
```

1. A1 GÓRA:

```
if a1.top.nn == targetNN AND a1.top.zlecenie == activeZl:  
    p = a1.pop()  
    currentPaleta.push(p.nn)  
    expectedInActiveIdx++  
    → continue
```

2. A2 GÓRA:

```
if a2.top.nn == targetNN AND a2.top.zlecenie == activeZl:  
    p = a2.pop()  
    currentPaleta.push(p.nn)  
    expectedInActiveIdx++  
    → continue
```

3. A1 DÓŁ (tasowanie A1→A2):

```
if a1.bottom.nn == targetNN AND a1.bottom.zlecenie == activeZl:  
    if len(a1) > 1:  
        topA1 = a1.pop()  
        if len(a2) >= maxA2: → BLOKADA  
        a2.push(topA1)      → log: "≠ Tasowanie: A1 góra → A2"  
        p = a1.shift()      (pobierz dół)  
        currentPaleta.push(p.nn)  
        expectedInActiveIdx++  
        → continue
```

4. A2 DÓŁ (tasowanie A2→A1):

```
if a2.bottom.nn == targetNN AND a2.bottom.zlecenie == activeZl:  
    if len(a2) > 1:  
        topA2 = a2.pop()  
        if len(a1) >= maxA1: → BLOKADA
```

```
        a1.push(topA2)           → log: "≠ Tasowanie: A2 góra → A1"
    p = a2.shift()               (pobierz dół)
    currentPaleta.push(p.nn)
    expectedInActiveIdx++
    → continue

→ Brak dopasowania: wyjdź z pętli
```

Po każdym dopasowaniu sprawdź czy `expectedInActiveIdx >= len(panels)` → jeśli tak, wywołaj **[6] SHIFT**.

Ważne: Tasowanie odbywa się wyłącznie wtedy, gdy bufor docelowy ma wolne miejsce. Jeśli jest pełny, procedura natychmiast zgłasza **BLOKADĘ**.

6. Zakończenie Palety – Shift (shiftOrders())

Wywoływane gdy `expectedInActiveIdx >= len(panels)` aktywnego zlecenia:

```
1. Zapisz { zlecenie, currentPaleta[] } do historyPalety[].
2. currentPaleta = []
3. activeOrderIdx = nextOrderIdx
4. nextOrderIdx++
5. expectedInActiveIdx = 0
6. Wywołaj recovery() ← panele z nowej aktywnej mogą już czekać w
```

7. Stany Końcowe i Wyniki Symulacji

Stan	Warunek	Komunikat
 ok	Wszystko ułożone / zbuforowane, bez odrzutów	PROCES PŁYNNY
 warn	rejectedCount > 0, ale nie ma blokady	ZAKOŃCZONO z odrzutami
 blocked	Oba bufony pełne lub blokada tasowania	BLOKADA STEROWNIKA

8. Schemat Przepływu (uproszczony)



9. Parametry Konfiguracyjne

Parametr	Domyślnie	Opis
maxA1	2	Pojemność bufora dalekiego (A1)
maxA2	2	Pojemność bufora bliskiego (A2)

Zwiększenie pojemności buforów redukuje ryzyko blokady kosztem większej złożoności tasowania.

10. Scenariusze Testowe (z Symulatora)

Nr	Nazwa	Opis
1	Idealna sekwencja	Wszystkie panele w kolejności – zero buforowania
2	Prosta zamiana	Dwa sąsiednie panele zamienione miejscami
3	Tasowanie	Panel z późniejszej pozycji przyjeżdża wcześniej, wymaga tasowania
4	Blokada (buf=2)	Panel brakujący przybywa jako ostatni – bufory się przepełniają
5	Większe bufory	Panele w odwrotnej kolejności, testuje głębokość tasowania
6	Reset palety	Ukończenie jednej palety i płynne przejście do następnej
7	Koniec zlecenia z brakami	Paleta zamknięta z-panelem, brakuje jednego panela ze środka